



**Aplicación
Configurable de
Seguros en Línea
Versión 10g
Manual de Estándares**

CONTENIDO

-
- 1. NOMENCLATURA
- 2. DEFINICIONES DE CAMPOS
- 3. DISEÑO DE PANTALLA
 - 3.1 ASPECTO GENERAL
 - 3.1.1 Pantalla Base
 - 3.1.2 Títulos, Descripciones y Leyendas
 - 3.1.3 Ventanas
 - 3.1.4 Campos
 - 3.1.5 Distribución de Títulos y Campos
 - 3.1.6 Secuencia de Campos
 - 3.1.7 Formatos de Campos
- 4. NAVEGACIÓN
 - 4.1 CONSIDERACIONES GENERALES
 - 4.1.1 Acciones Básicas
 - 4.1.2 Uso de Menús y Teclas Rápidas
 - 4.1.3 Definición de teclas de función en ORACLE
- 5. ORACLE DEVELOPER SUITE 10g FORMS BUILDER
 - 5.1 NOMENCLATURA
 - 5.1.1 Nombre de las Formas
 - 5.1.2 Nombre de los Bloques
 - 5.1.3 Nombre de los Campos
 - 5.1.4. Nombre de los TRIGGERS Definidos por el Usuario
 - 5.1.5. Nombre de las Tablas
 - 5.1.6. Nombre de los Procedimientos
 - 5.2. DOCUMENTACIÓN INTERNA
 - 5.2.1. En las Formas
 - 5.2.2. En los Bloques
 - 5.2.3. En los Campos
 - 5.2.4. En los TRIGGERS
 - 5.2.5. En los Procedimientos
 - 5.3. MENSAJES
 - 5.4. ATRIBUTOS DE LOS OBJETOS DE ORACLE DEVELOPER SUITE 10g FORMS BUILDER
 - 5.4.1. A nivel de la Forma
 - 5.4.1.1. Título de la forma
 - 5.4.1.2. Validation Unit
 - 5.4.2. A nivel del Bloque
 - 5.4.2.1. Records Buffered
 - 5.4.2.2. Array size
 - 5.4.2.3. Primary Key
 - 5.4.3. A nivel de los campos
 - 5.4.3.1. General



- 5.4.3.2. Secuencia de los campos
- 5.4.3.3. Data Type
- 5.4.3.4. Field Length
- 5.4.3.5. Display Length
- 5.4.3.6. Query Length
- 5.4.3.7. Base Table
- 5.4.3.8. Primary Key
- 5.4.3.9. Upper Case
- 5.4.3.10. Campos de selección SI/NO
- 5.4.3.11. Automatic Skip
- 5.4.3.12. Automatic Hint
- 5.4.3.13. Enforce Foreign Key
- 5.4.3.14. Format Mask
- 5.4.3.15. List Of Values
- 5.5. **TRIGGERS**
 - 5.5.1. Fallas en los TRIGGERS
 - 5.5.2. Sentencias SQL Validas
 - 5.5.3. TRIGGERS de Usuario
 - 5.5.4. TRIGGERS de Teclas
 - 5.5.5. TRIGGERS de Navegación
 - 5.5.5.1. ON-NEW-RECORD
 - 5.5.5.2. PRE y POST-COMMIT
 - 5.5.5.3. PRE-QUERY
 - 5.5.5.4. POST-CHANGE
 - 5.5.5.5. POST-QUERY
 - 5.5.6. Validación en los TRIGGERS
 - 5.5.7. Eliminación de Registros
 - 5.5.8. Generar Claves Únicas en las Tablas.
- 5.6. **VARIABLES**
 - 5.6.1. Uso de Variables en PL/SQL
 - 5.6.2. Uso de Variables Globales
- 5.7. **VIEWS**
- 5.8. **TABLA DUAL**
- 5.9. **CODIFICACIÓN PL/SQL**
- 5.10. **CURSORES**



1. Nomenclatura

- Las palabras significativas (no interjecciones) de todos los títulos se deben escribir con la primera letra en mayúscula y el resto en minúscula.
- Todas las abreviaciones que se utilicen en el sistema deberán incluirse en el anexo "A" - Abreviaciones Estándar.
- Las abreviaciones de uso interno son de carácter obligatorio para todo lo referente a diseño y programación, p. Ej. entidades, atributos, procesos, etc. Excepto cuando los términos se usen individualmente, en cuyo caso el uso de la abreviación será opcional.

Las abreviaciones para títulos sólo se utilizarán cuando sea necesario.

Las abreviaciones de títulos estarán separadas de otras abreviaciones o palabras por un punto (.), sin espacios en blanco.

p. Ej. Cod.Cobert, Num.Póliza, F.Emisión

Los nombres de campos de la base de datos se compondrán por las abreviaciones o términos correspondientes sin ningún tipo de separador. Cada abreviación o término tendrá la primera letra en mayúscula y el resto en minúsculas.

p.Ej. NumPol, CodTerminacion, FecEmi

Los nombres de las variables que correspondan a campos de la base de datos, deberán llamarse como el campo asociado, precediéndolo con una letra minúscula que indique el tipo de la variable.

p.Ej. nNumPol, cCodTerminacion, dFecEmi

Los prefijos que se utilizarán son los siguientes:

Cara
cter

Num
érico

Fech
a
(Dat
e)

Lógi
co

- Los nombres de variables que no hagan referencia a la base de datos se compondrán por el prefijo correspondiente al tipo, un carácter de "subrayado" y el o los términos correspondientes al significado de la variable, según la convención indicada anteriormente.

p.Ej. c_TipoBloque, d_FecReferencia



- Los nombres de tablas y entidades deberán ir en mayúsculas y se utilizará el caracter de "subrayado" como separador.

p.Ej. RAMO, CERT, COBERT_CERT





2. Definiciones de campos

- Los tipos y longitudes estándar para las clases de campos principales son las siguientes:

Clase	T	E	D	(dB ASE /UI 2)	O
Suma Aseg.		1	2	15,2	1
Primaria		1	2	15,2	1
Comisión		1	2	14,2	1
Total		1	2	17,2	1
Tasa			6	10,6	
Porcentaje			4	8,4	
Nro. Póliza		1		10,0	1
Nro. Recibo		1		10,0	1
Nro. Certificado		1		10,0	1

- Los campos que se utilicen como indicadores serán alfanuméricos de una posición y el nombre debe comenzar con el prefijo *Ind.*

Los valores válidos son 'S' y 'N'

- Los campos que formen parte de la clave de las tablas deberán ser tipo carácter, excepto cuando se requiera llevar un contador de las ocurrencias de registros, en cuyo caso serán numéricos. p.Ej. la póliza, el recibo y el certificado son numéricos.
- Los campos de estatus deben ser tipo carácter de tres posiciones y el nombre debe comenzar con el prefijo *Sts.*

El valor de los estatus debe ser nemotécnico,

p. Ej. CFG: Configuración, PRD: Producción

3. Diseño de pantalla

3.1 ASPECTO GENERAL

- El diseño de las pantallas debe ser agradable a la vista y amigable ante el usuario. Se deben observar algunos puntos de carácter general:
- - Las pantallas deben estar "balanceadas".
 - No deben verse atiborrada de información.
 - Deben tener un "look and feel" estándar.

3.1.1 Pantalla Base

- El área libre de trabajo en la pantalla debe tener un tamaño máximo de 589 de ancho y 312 de alto o maximizado. El área de la pantalla debe contener el Toolbar vertical.
- La línea inferior está reservada para los mensajes de ORACLE.

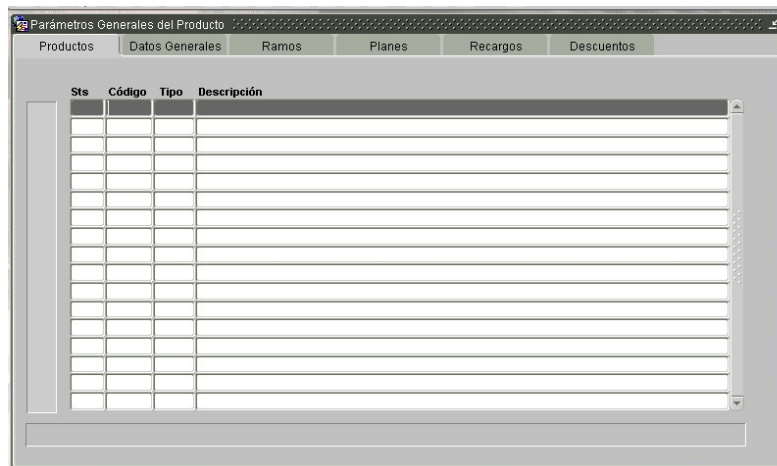


Fig. 3.1.1 Pantalla Base

3.1.2 Títulos, Descripciones y Leyendas

- Los títulos y descripciones se deben mantener consistentes en todo el sistema. Se deben utilizar los nombres de entidades y atributos cuando sea apropiado.

- Las palabras significativas de los títulos y descripciones se deben escribir con la primera letra en mayúscula y el resto en minúsculas, y la fuente a utilizar debe ser Ms Sans Serif negrita de 8 inch.
- Las abreviaciones válidas de términos están indicadas en la sección de NOMENCLATURA de este manual.
- Las leyendas, mensajes y texto que representen oraciones completamente desarrolladas, se escribirán en minúsculas, a excepción de los nombres propios y la primera palabra del texto, que usarán la primera letra en mayúscula.

p.Ej.

Se deben especificar cláusulas para este ramo ?

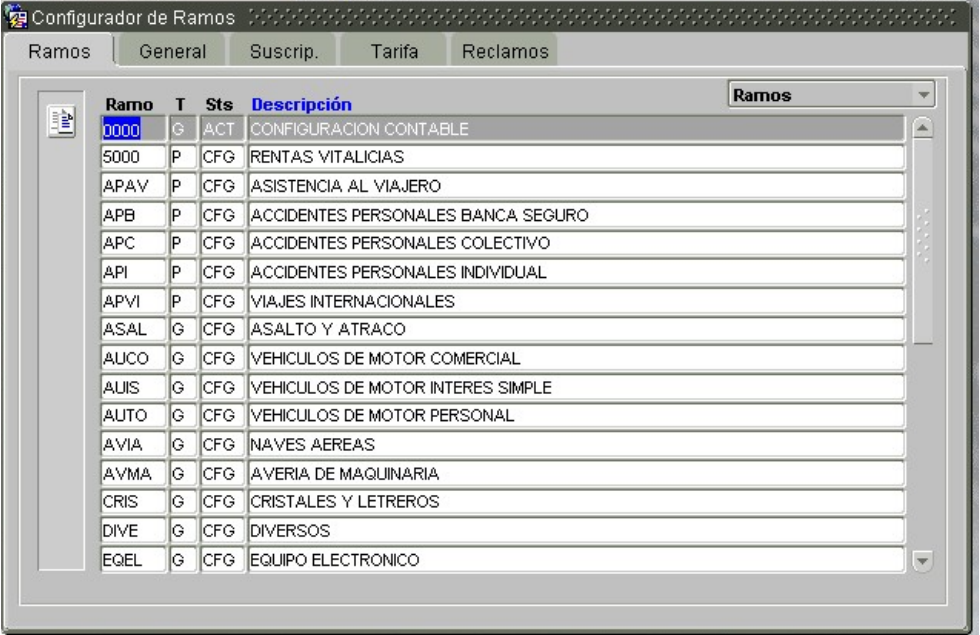
- Cuando los títulos representen alternativas, estas se deben separar con " / " (espacio-barra-espacio).

p.Ej.

Sucursal / Oficina

3.1.3 Ventanas

- Toda información que se presente en el área libre de la pantalla deberá estar delimitada por un marco de línea simple (ventana).
- Toda ventana debe tener un título en el borde superior, alineado hacia la izquierda.
- Dentro de las ventanas se debe mantener un margen lateral de al menos un carácter, entre el contenido de la ventana y el borde.
- Las ventanas donde se presentan varios registros en forma columnar, se denominan "Ventanas tipo LISTA" (Fig 3.1.2).



The screenshot shows a software window titled 'Configurador de Ramos'. It has a tabbed interface with 'Ramos' selected. Below the tabs is a table with the following data:

Ramo	T	Sts	Descripción
0000	G	ACT	CONFIGURACION CONTABLE
5000	P	CFG	RENTAS VITALICIAS
APAV	P	CFG	ASISTENCIA AL VIAJERO
APB	P	CFG	ACCIDENTES PERSONALES BANCA SEGURO
APC	P	CFG	ACCIDENTES PERSONALES COLECTIVO
API	P	CFG	ACCIDENTES PERSONALES INDIVIDUAL
APVI	P	CFG	VIAJES INTERNACIONALES
ASAL	G	CFG	ASALTO Y ATRACO
AUCO	G	CFG	VEHICULOS DE MOTOR COMERCIAL
AUIS	G	CFG	VEHICULOS DE MOTOR INTERES SIMPLE
AUTO	G	CFG	VEHICULOS DE MOTOR PERSONAL
AVIA	G	CFG	NAVES AEREAS
AVMA	G	CFG	AVERIA DE MAQUINARIA
CRIS	G	CFG	CRISTALES Y LETREROS
DIVE	G	CFG	DIVERSOS
EQEL	G	CFG	EQUIPO ELECTRONICO

Fig. 3.1.2 Ventana

- Las ventanas donde se presentan varios campos de un sólo registro a la vez, se denominan "Ventanas tipo FICHA" (Fig 3.1.3).

Fig. 3.1.3 Distribución de Títulos y Campos

3.1.4 Campos

- En las ventanas tipo LISTA no editables, se debe presentar todos los campos del registro en video inverso.

Los Campos no editables deben ser de tipo display, y si se refiere a una descripción de un código debe ser de color gris.

3.1.5 Distribución de Títulos y Campos

- Los títulos y descripciones de los campos simples (no columnares) deben ir a la izquierda de estos, separados por un espacio en blanco.

p.Ej. Código Cliente XXXXXX

- Cuando el tamaño del campo impida lo anterior, el título se ubicará en la línea inmediata superior. La alineación de los títulos seguirá la siguiente norma:

- Campo numérico, título alineado a la derecha.
- Campo porcentaje, "%" sobre el punto decimal. Si el "%" tiene calificativo y no se puede alinear el "%" sobre el punto decimal, se alinea todo el título a la derecha.
- Para el resto de los campos, los títulos se alinean a la izquierda.



p.Ej.

Nombre Cliente	del	% I n t	Monto Crédito
XXXXXXXXXXXXXXXX		9	999,999,9
XXXXXX		9	99.99
		.	
		9	
		9	
		9	
		9	

- Los títulos de campos columnares se deben ubicar tal como se indica en el párrafo anterior.

p.Ej.

Nombre Cliente	del	% I n t	Monto Crédito
XXXXXXXXXXXXXXXX		9	999,999,9
XXXXXX		9	99.99
		.	
		9	
		9	
		9	
		9	
XXXXXXXXXXXXXXXX		9	999,999,9
XXXXXX		9	99.99
		.	
		9	
		9	
		9	
		9	
XXXXXXXXXXXXXXXX		9	999,999,9
XXXXXX		9	99.99
		.	
		9	
		9	
		9	
		9	

- Los campos relacionados se deben agrupar creando, si es necesario, secciones dentro de la ventana, delimitadas por líneas continuas.
- Las secciones pueden tener títulos, que deben estar alineados con el contenido de la sección.
- Los campos y títulos se deben alinear en forma de bloques, siguiendo la siguiente norma:
 - Los títulos se deben alinear a la izquierda.
 - Los campos numéricos se deben alinear por el punto decimal.
 - El resto de los campos se debe alinear a la izquierda.



- Los totales de campos columnares deben ubicarse en la línea inmediata inferior a la columna totalizada.
- Los campos de totalización de columnas debe llevar el título de "Total" mas un calificativo, si es necesario, a la izquierda del campo y alineado a la derecha, con respecto a la columna anterior a la totalizada.

- Los campos columnares deben estar separados sólo por espacios en blanco (Fig. 3.1.4)

Fig. 3.1.4 Campos Columnares

3.1.6 Secuencia de Campos

- La secuencia de los campos debe mantener la lógica de operación.
- Manteniendo las restricciones de la secuencia lógica, los campos se deben ordenar como sigue:
 - Primero los campos obligatorios.
 - Luego los campos opcionales que se actualizan frecuentemente.
 - Finalmente los campos opcionales que se actualizan ocasionalmente.
- Los campos opcionales se pueden ubicar en ventanas separadas.

3.1.7 Formatos de Campos

- El formato estándar para las fechas será DD/MM/AAAA.
- Los campos numéricos deben tener separadores de miles, millones, etc.
- Los campos correspondientes a indicadores (Si / No) serán de un sólo carácter, para reducir el trabajo del usuario.
- Los campos deben tener tooltips y hints que ayuden al usuario al momento de ingresar información
- Se debe utilizar el TAB en la navegación de las formas
- El scrollbar debe estar ubicado a la izquierda



4. Navegación

4.1 CONSIDERACIONES GENERALES

- El objetivo principal de todos los aspectos de navegación, es facilitar la ejecución de todos los procesos por parte de los usuarios que no estén familiarizados con la operación del sistema.
- Al mismo tiempo se debe agilizar el trabajo de los usuarios que conozcan bien la aplicación, mediante el uso de teclas rápidas para las opciones de uso frecuente. Las teclas rápidas estarán documentadas en la pantalla de ayuda de teclas de FORMS.
- El concepto fundamental de la navegación es presentar la información partiendo de lo general para luego ir a lo particular. Debe facilitar la ubicación en el contexto del proceso, para luego ir a los aspectos específicos.

4.1.1 Acciones Básicas

- El acceso inicial a todas las tablas del sistema debe ser a través de una ventana tipo LISTA (campos columnares).
- Cuando la cantidad de información de una tabla lo requiera, se utilizará una ventana tipo FICHA (puede ser multipágina), activada desde la ventana tipo LISTA. En este caso, la ventana tipo LISTA correspondiente no será editable.
- Cuando se utilicen ventanas tipo FICHA, todas las operaciones de mantenimiento se realizarán sobre éstas.

4.1.2 Uso de Menús y Teclas Rápidas

- Desde cualquier pantalla se debe poder regresar al paso anterior del proceso o cancelar todo el proceso que se esté ejecutando en el momento.
- En toda pantalla debe haber dos formas de realizar las operaciones:

Activando un menú POP-UP que de acceso a todas las opciones asociadas a la pantalla.

Con Botones de acceso rápido para las operaciones de uso frecuente
- La tecla NXTBLK siempre debe ejecutar el próximo paso del proceso.



- Cuando el próximo paso del proceso implique varias opciones, se debe presentar un menú POP-UP con todas las opciones disponibles.

4.1.3 Definición de teclas de función en ORACLE

Especificación de caracter no válida	TECLA	"PROCEDURE" POR OMISION	DESCRIPCION
--------------------------------------	-------	-------------------------	-------------

Función original de ORACLE -
Teclas sin modificación

Aceptar	KEY-COMMIT	COMMIT_RECORD	Actualiza los registros en la base de datos
Duplicar Registro	KEY-DUPREC	DUPLICATE_RECORD	Duplicar valores del registro anterior
Duplicar Campo	KEY-DUPFLD	DUPLICATE_FIELD	Duplicar valor del mismo campo del registro anterior
Editar en Ventana	KEY-EDIT	EDIT_FIELD	Editar un campo en ventana "Pop-Up". Si FIELD LENGTH > DISPLAY LENGTH
Ejecutar Consulta	KEY-EXEQRY	EXEQUTE_QUERY	Ejecutar la Consulta especificada con KEY-ENTQRY
Campo Anterior	KEY-PRVFLD	PREVIOUS_FIELD	Campo anterior de la pantalla

Función original de ORACLE -
Teclas redefinidas para todo el Sistema

Próximo Registro	KEY-DOWN	NEXT_RECORD	Cursor abajo. Proximo registro
Registro Anterior	KEY-UP	PREVIOUS_RECORD	Cursor arriba. Registro anterior
Ayuda	KEY-HELP	AYUDA	Ayuda. Sistema de ayuda al usuario

Función específica del Sistema -
Teclas redefinidas según el proceso, cuando sea necesario

Terminar Proceso	KEY-EXIT	SALIDA	Terminar el proceso en ejecución. Salida de la forma. Presenta pantalla para confirmar el COMMIT
Próximo Campo	KEY-NXTFLD	NEXT_FIELD	Próximo campo de la pantalla
Próximo Paso	KEY-NXTBLK	NULL	Próximo paso del proceso que se ejecuta
Paso Anterior	KEY_PRVBLK	NULL	Paso anterior del proceso que se ejecuta
Incluir Registro	KEY-CREREC	CREATE_RECORD	Crear (insertar) un registro en la tabla
Eliminar Registro	KEY-DELREC	DELETE_RECORD	Eliminar registro. Pantalla de confirmación
Iniciar Consulta	KEY-ENTQRY	NULL	Entrar en modo de consulta

Función específica del Sistema -
Teclas redefinidas según el
proceso, cuando sea necesario

Menú	KEY-MENU	NULL	Menú del operador. Presenta un menú "Pop-Up" para ejecutar procesos que no tienen tecla rápida asociada.
Lista de Valores	KEY-LISTVAL	LIST_VALUES	Lista de valores, reprogramable a nivel de campo.
Imprimir	KEY-PRINT	NULL	Imprimir. En algunas aplicaciones se debe ejecutar un reporte "formateado"
Próxima Página	KEY- SCRDWN	SCROLL_DOWN	Pagineo. Se redefine en formas multi-página para presentar la próxima página.
Página Anterior	KEY-SCRUP	SCROLL_UP	Igual KEY- SCRDWN pero con la pagina anterior.

Función no asignada - Teclas
redefinibles libremente según lo
requiera el proceso

	KEY-CLRBLK	NULL	Limpiar bloque
	KEY-CLRFRM	NULL	Limpiar forma
	KEY-CLRREC	NULL	Limpiar registro
	KEY-CQUERY	NULL	Conteo de registros
	KEY-UPDREC	NULL	Bloquear registro para actualización
	KEY-NXTKEY	NULL	Proximo campo de la clave primaria
	KEY-NXTREC	NULL	Proximo registro
	KEY-PRVREC	NULL	Registro anterior
	KEY-NXTSET	NULL	Proximo grupo de datos





5. ORACLE DEVELOPER SUITE 10g FORMS BUILDER

5.1. NOMENCLATURA

5.1.1. Nombre de las Formas

- Los nombres de las formas deben estar en Mayúsculas

5.1.2. Nombre de los Bloques

- El nombre de los bloques debe ser en el formato "B99_X", esto es, el nombre del bloque siempre comienza con B y a continuación el número de la ventana según el esquema de navegación correspondiente.
- A continuación los bloques se numeran en forma secuencial, pero respetando siempre el número de la ventana correspondiente, p.Ej.: B01_1, B01_2, etc.
- Los bloques de consulta asociados a una ventana se llamarán "Bxx_Q", donde "xx" representa el número de la ventana correspondiente.
- El nombre de los bloques, incluidos los bloques virtuales :GLOBAL y :SYSTEM, debe estar en mayúsculas.

5.1.3. Nombre de los Campos

- El nombre de los campos y variables se debe ajustar a lo indicado en el punto 1 (NOMENCLATURA) de este manual.
- Adicionalmente a esto, los nombres de los campos de la pantalla deben estar precedidos siempre con el nombre del bloque, siguiendo la sintaxis de FORMS (:NOMBRE_DEL_BLOQUE.Nombre_del_campo), esto evitará posibles problemas en el mantenimiento si se agregan nuevos campos y / o nuevos bloques a la forma.

5.1.4. Nombre de los TRIGGERS Definidos por el Usuario

- En principio se deben utilizar PROCEDURES en vez de TRIGGERS de usuario. En todo caso el nombre utilizado debe ser lo suficientemente claro que indique su función.
- Todo nombre de TRIGGERS debe estar en mayúsculas.

5.1.5. Nombre de las Tablas

- Los nombres de las tablas deben coincidir con la especificación del DESIGNER.
- En todas las referencias a tablas del sistema, se debe usar el nombre propio de estas. Nunca se debe utilizar precedido con el nombre del dueño.
- Los nombres de tablas deben estar en mayúsculas.

5.1.6. Nombre de los Procedimientos

- 
- El nombre de los procedimientos debe ser lo suficientemente claro que explique la función principal del mismo.
 - Los nombres de los procedimientos deben estar en mayúsculas.

5.2. DOCUMENTACIÓN INTERNA

5.2.1. En las Formas

- Todas las formas deben estar documentadas, utilizando la sección Comment en las propiedades de la forma, de la siguiente manera:

Forma:	Nombre completo de la aplicación
Autor:	Nombre del programador
Versión:	Numero de la versión de la aplicación
Fecha Prog:	Fecha en que se programó originalmente
Descripción:	Descripción de la aplicación, propósito, funciones (4 líneas máximo).
Modificación:	Cada modificación debe incluir: fecha, autor y descripción.

5.2.2. En los Bloques

- Todo bloque debe contener una descripción que indique el propósito y funcionamiento del bloque.

5.2.3. En los Campos

- Todo campo que no este asociado a una tabla o que no se despliegue en pantalla debe ser comentado para explicar su propósito. Se asume que los campos desplegables asociados a una tabla su propósito es obvio y no es necesario documentarlo.

5.2.4. En los TRIGGERS

- Todo aquel TRIGGERS propio de FORMS que no realice la función asociada a él y todo TRIGGERS definido por el programador debe ser documentado en el texto del mismo.
- También se debe documentar también cada paso que se realice dentro del TRIGGERS.
- Sobre la programación de los TRIGGERS de usuario, refiérase al punto 5.1.4.

5.2.5. En los Procedimientos

- La primera línea de un procedimiento debe contener, en comentarios, el nombre descriptivo del mismo. Esto con el objetivo que en la pantalla columnar donde se revisan los procedimientos de FORMS, se pueda leer el nombre y la descripción sin tener que "abrir" la pantalla.

5.2.6. En los Procedimientos



- Si el procedimiento esta en una forma referencial, adicionalmente debe contener una sección que indique:

Autor:
Fecha de creación:
Versión:

y por cada modificación se debe indicar la Fecha, Descripción y el Autor de la modificación.

- Por cada bloque de operación de PL/SQL dentro de un procedimiento debe ser documentado de manera que se indique el propósito y la lógica básica de la operación que se realiza. Nunca se debe comentar líneas simples de PL/SQL.

5.3. MENSAJES

- Todos los mensajes de error se manejan a través del sistema de mensajes diseñado para las aplicaciones de FORMS.
- Es decir, todo mensaje debe ser desplegado utilizando la función pre-programada `PR_MENSAJE` (ver anexo de documentación de funciones para el manejo de esta función).
- Los mensajes de error propios de la aplicación deben estar codificados siguiendo la siguiente estructura:

```
X          99999          XXX XXX
          XXX
|
|
|
|          |          Parámetros del Mensaje
|          |          Código del error
|          |
|          |          Origen del error (FRM:Forms, ORA:Oracle, AIF =
|          |          FORMS USUARIO ABD Base de Datos del Usuario)
```

- En ciertas aplicaciones es recomendable no desplegar los mensajes de información propios de FORMS como:
"Transaction complete -- records posted and committed"
o su equivalente en el sistema de mensajes, para esto se deben utilizar la variable del sistema `:SYSTEM.MESSAGE_LEVEL` asignándole un valor de 5 antes de realizar la operación que genera el mensaje y asegurándose de asignarle el valor de 0 antes de salir de la rutina.
- Se deben programar los TRIGGERS ON-MESSAGE y ON-ERROR a nivel de la forma según aparece en la forma MODELO.



5.4. ATRIBUTOS DE LOS OBJETOS de ORACLE DEVELOPER SUITE 10g FORMS BUILDER

5.4.1. A nivel de la Forma

5.4.1.1. Titulo de la forma

- Debe contener el nombre corto de la aplicación tal como se especificó en el punto 5.1.

5.4.1.2. Validation Unit

- Debe mantener el valor asignado por omisión al generar la forma. Solo se debe cambiar en condiciones especiales

5.4.1.3. Firts Navigation Block

- Debe contener el primer bloque navegable al ejecutarse la forma

5.4.1.4. Console Windows

- Primera ventana a mostrar la forma

5.4.1.5. Menu Module

- Debe contener el menú estándar de Acsel/X (MNUDEFA)

5.4.2. A nivel del Bloque

5.4.2.1. Current Record Attribute

- Este valor indica el atributo visual a presentar los campos del bloque en la pantalla, debe utilizarse en forma referencia el contenido en la forma FRMACSEL (STD_VISUAL_ATTRIBUTE), solo cuando el tipo de pantalla sea tipo lista.

5.4.2.2. Next Navigation Block

- Próximo bloque a navegar

5.4.2.3. Previous Navigation Block

- Previo bloque navegable en la forma, si es el primer bloque debe indicarse el mismo.

5.4.2.4. Records Buffered

- Este valor indica el numero de registros que se mantienen en memoria cuando se realiza un QUERY a la base de datos. El valor no debe ser menor al de ARRAY SIZE (punto siguiente) y debe estar relacionado con la cantidad de registros que se muestran a la vez en pantalla.

5.4.2.5. Array size

- Este valor indica la cantidad de registros seleccionados cada vez que se ejecuta el Query. Debe ser menor que el valor de Records Buffered y mayor a la cantidad de registros que se despliegan a la vez en la pantalla.

5.4.2.6. Primary Key

- Esta facilidad existe solo para mantener compatibilidad con versiones anteriores de ORACLE DEVELOPER SUITE 10g FORMS BUILDER. No se debe utilizar para garantizar la no-duplicidad de los registros.
- La mejor manera de hacer esto es programando TRIGGERS ON-VALIDATE-FIELD y / o PRE-INSERT que chequeen la unicidad del registro apoyándose en la definición del Primary Key de la tabla.

5.4.3. A nivel de los campos

5.4.3.1. General

- Donde sea posible es preferible dejar que FORMS realice el trabajo de iniciación (default Value) y validación de campos utilizando lo especificado en la definición de los mismos (Data Type, Default Value, etc.).

5.4.3.2. Secuencia de los campos

- Los campos que no se despliegan deben estar después de los que se despliegan. Esto garantiza que FORMS no navegue por campos innecesarios que pueden traer complicaciones de validación.

5.4.3.3. Data Type

- Este atributo determina el tipo de validación que se realiza sobre el contenido del campo.
- Los siguientes "Data Types" existen en ORACLE DEVELOPER SUITE 10g FORMS BUILDER solo por mantener la compatibilidad pero deben ser excluidos en la programación de nuevas aplicaciones: RINT, RMONEY y RNUMBER.

5.4.3.4. Maximum Length

- El largo de los campos asociados a una tabla debe mantener lo especificado en la tabla, con excepción de los campos numéricos, donde se agrega un espacio para el punto decimal y otro para el signo en el caso que se utilice y en aquellos casos de campos numéricos con formato de edición.

5.4.3.6. Query Length

- El Query Length puede ser mayor al Maximum Length pero solo se debe utilizar más grande en el caso que el campo sea numérico para que el usuario pueda utilizar operadores (mayor, menor, etc.)
- En general el Query Length debe ser igual al Maximum Length.

5.4.3.7. Base Table Item

- Se debe utilizar solo en los campos que pertenecen a la tabla asociada al bloque.

5.4.3.8. Primary Key

- Solo debe ser utilizada como documentación y nunca en unión con la definición de Unique Key del Bloque .

5.4.3.9. Case Restriction

- Todos los campos que pertenecen a la clave principal de la tabla o que forman parte de una clave foránea deben ser declarados Upper Case para evitar complicaciones en las búsquedas posteriores en la tabla.
- Los campos alfanuméricos que no cumplen esta condición, como descripciones, direcciones, pueden ser declarados no Upper Case.

5.4.3.10. Campos de selección SI / NO

- Estos campos deben ser definidos siempre como CHAR(1) y validados para sólo aceptar 'S' o 'N'.

5.4.3.11. Auto Skip

- Esta opción se debe dejar siempre deshabilitada, para que sea necesario el <ENTER> para pasar al próximo campo, aunque se llene por completo el actual.

5.4.3.12. Autoc Hint

- Se debe utilizar en todos aquellos campos donde el título en pantalla no aclare lo suficiente al usuario sobre el dato a ingresar y en aquellos que se espera un rango de valores específico.

5.4.3.13. Enforce Foreign Key

- Si el campo utiliza esta facilidad debe deshabilitarse el atributo de Input Allowed de manera que el campo mantenga el valor ingresado en el Foreign Key especificado.

5.4.3.14. Format Mask

- No se debe utilizar formato de edición de manera e poder utilizar la facilidad de "lenguaje" de ORACLE para que se adapte automáticamente a los distintos formatos existentes de fechas y números.

5.4.3.15. LOV

- Debe utilizarse esta facilidad donde sea posible. Hay que tener en cuenta que esta función siempre cambia el valor del campo asociado a la selección, aunque el usuario no haya realizado una.
- Los valores que se muestran deben estar ordenados.
- Es preferible mostrar primero los valores que le sean más familiares al usuario como la descripción en vez del código. Al hacer esto es necesario crear un campo "DUMMY" para aceptar el valor de la descripción seleccionada antes del campo donde se va a colocar el valor del código

- Hay que tener en consideración que el **List Of Values** no es totalmente eficiente en las búsquedas a la base de datos, por lo tanto debe estar restringido a la búsqueda en tablas con pocos registros. En el caso que la tabla contenga muchos registros y se requiera realizar ayudas tipo Lista de Valores es preferible crear formas que sean llamadas utilizando CALLQUERY y que retornen los valores deseados a la forma original.
- En los casos de listas de valores con selección múltiple, existen unas formas ya programadas para que sean utilizadas en las aplicaciones que así lo necesiten (Ver anexo de documentación de las formas pre-programadas).

5.5. TRIGGERS

5.5.1. Fallas en los TRIGGERS

- Los TRIGGERS fallan por dos razones:
 - a) Se ejecuta la excepción `FORM_TRIGGER_FAIULRE` y en ese caso se debe informar al usuario con un mensaje claro por pantalla (Ver punto 3 anterior).
 - b) Se produce una excepción no prevista en el TRIGGER (EXCEPTION) en este caso se debe también mostrar un mensaje al usuario indicándole que ocurrió un error no previsto con el correspondiente numero de error para que sea reportado.

5.5.2. Sentencias SQL Validas

- Las sentencias `SELECT` son siempre validas en los TRIGGERS.
- Sentencias de DDL (Data Definition Language) no son permitidas.
- A pesar que sentencias de DML (Data Manipulation Language) son permitidas en cualquier TRIGGER, es recomendable que se tengan en consideración los siguientes puntos antes de incluirlos en un TRIGGER distinto a los existentes para la actualización de la base de datos:
 - Hay que tener una buena razón para hacerlo.
 - Informe al usuario, con mensajes apropiados, que la aplicación ha realizado una actualización.
 - Nunca escriba una rutina de actualización sobre la tabla base del bloque.
 - Prevea la posibilidad que algunos cambios no sean actualizados al ejecutarse el TRIGGER de salida de la forma (`EXIT_FORM`).

5.5.3. TRIGGERS de Usuario

- Como ya se indicó, procure no utilizar TRIGGERS de usuario, es preferible que se programen Procedures.
- La razón principal es que la manera de abortar un TRIGGER es bajo el mecanismo de `RAISE FORM_TRIGGER_FAILURE`.

- Si esto se ejecuta en un TRIGGER de usuario, la condición de falla no es transmitida automáticamente al TRIGGER que lo llamó, por lo que este también tiene que hacer un chequeo de la condición de error. En el caso de PROCEDURES esto no pasa y la condición de error si es transmitida al TRIGGER que invoco al procedimiento.

5.5.4. TRIGGERS de Teclas

- Como parte de estos estándares, se tienen clasificadas las TRIGGERS de teclas en:
 - a) Función original de ORACLE - Teclas sin modificación.
 - b) Función original de ORACLE, Teclas redefinidas para todo el sistema.
 - c) Función específica del Sistema - Teclas redefinidas según el proceso, siempre que sea necesario.
 - d) Función no asignada - Teclas redefinibles libremente según lo requiera el proceso.

Ver punto 4.1.3, Definición de teclas de función.

- Todos aquellos TRIGGERS de teclas que hayan sido programados, deben mantener un funcionamiento similar en toda la aplicación. Esto facilitará la operación a los usuarios, al poder memorizar la secuencia de teclas necesarias para realizar operaciones similares.
- Cualquier TRIGGER de Tecla que sea programado debe contener una descripción corta de su funcionamiento, en mayúsculas, en el campo Description de la pantalla de definición de TRIGGERS, para informar al usuario cuando éste presione la tecla de `SHOW KEYS`.

5.5.5. TRIGGERS de Navegación

- Los TRIGGERS de navegación, que incluyen: PRE-FORM, POST-FORM, PRE-BLOCK, POST-BLOCK, PRE-RECORD, PRE-FIELD Y POST-FIELD no deben ser utilizados con fines de validación porque de fallar esta, puede resultar en un FATAL ERROR en la aplicación.
- Al ejecutar un COMMIT_FORM, se activan los TRIGGERS de PRE y POST FIELD y BLOCK. Si alguno de ellos falla, entonces el TRIGGER de COMMIT también fallará.

5.5.5.1. KEY-CREREC

- Este TRIGGER se activa cada vez que el cursor se mueve a un registro nuevo. Este TRIGGER se puede utilizar para rellenar con datos por Omisión al nuevo registro.

5.5.5.2. PRE y POST-COMMIT

- Estos TRIGGERS se activan una sola vez por cada actualización que se realice, sin importar el numero de registros que se



actualicen. Son activados tanto por TRIGGERS COMMIT como por POST.

5.5.5.3. PRE-QUERY

- Los TRIGGERS PRE-QUERY se utilizan para validar la entrada del QUERY del usuario y proveer funciones adicionales antes de ejecutar el QUERY.
- Para evitar la ejecución de QUERIES que pudieran retornar una gran cantidad de registros o que no utilizan los campos claves definidos para la tabla, se debe escribir un TRIGGER similar al siguiente:

```
PRE-QUERY
IF :CodCliente IS NULL AND :CodCobert IS NULL THEN
    MESSAGE(PR_MENSAJE('AIF',99999,' ','Cliente',
        'Cobertura'));
    RAISE FORM_TRIGGER_FAILURE;
END IF;
```

5.5.5.4. POST-CHANGE

- En general se debe evitar el usar TRIGGERS POST-CHANGE, es recomendable utilizar una combinación de TRIGGERS ON-VALIDATE-FIELD y POST-QUERY. Esto distingue el procesamiento a realizar entre los registros traídos de la tabla a los registros creados o actualizados luego. Específicamente esto permite:
 - a) Evitar la validación de los registros obtenidos en el QUERY.
 - b) Permite manejar validaciones específicas para cada momento (Luego del QUERY y luego de un cambio en el registro).

5.5.5.5. POST-QUERY

- Los TRIGGERS POST-QUERY se utilizan para realizar cálculos, completar datos, etc. por cada registro que se va a mostrar en pantalla. Nunca se despliegan mensajes en un TRIGGER POST-QUERY debido a que por cada registro que se lea de la tabla va a aparecer el mismo mensaje en pantalla con la agravante que el usuario debe presionar ENTER para pasar cada mensaje.

5.5.6. Validación en los TRIGGERS

- No se deben utilizar los TRIGGERS de navegación (NEXT y PREVIOUS FIELD/RECORD/BLOCK) para realizar validaciones. Para esto se deben utilizar los TRIGGERS ON-VALIDATE-FIELD y ON-VALIDATE-RECORD.

5.5.7. Eliminación de Registros

- Las validaciones antes de eliminar un registro se deben realizar en el TRIGGER KEY-DELREC.

Como es posible que la situación de un registro cambie entre que el usuario presiona la tecla de eliminación y que se hace el COMMIT, es recomendable programar el TRIGGER de validación de la eliminación en un TRIGGER a nivel de la forma o en procedure invocado tanto por la tecla KEY-DELREC como por el TRIGGER PRE-DELETE. Se recomienda el segundo enfoque, según lo indicado anteriormente.

Antes de realizar el DELETE_RECORD, se debe chequear el estatus del registro con la variable :SYSTEM.RECORD_STATUS, si este tiene valor 'NEW' o 'INSERT' no es necesario eliminar el registro sino que simplemente se realiza un CLEAR_RECORD.

5.5.8. Generar Claves Únicas en las Tablas.

- En algunos casos es preferible utilizar campos que contengan secuencias numéricas para definir la clave primaria de una tabla en vez de utilizar campos propios editables de la misma.
- Existen 2 métodos para generar números no duplicados. El primero se basa en la utilización de los SEQUENCES manejados automáticamente por ORACLE y pueden ser utilizados en los valores "Default Value" en el campo relacionado así:

```
Default Value: sequence.nombre.nextval
```

- La ventaja de este método es que es manejado por ORACLE y no necesita programación adicional. Las desventajas son que ORACLE realiza un SELECT por cada numero que se genera, y ORACLE genera un numero cada vez que el cursor llega a un NEW-RECORD y de esta manera pueden quedar "espacios" entre la numeración.
- Utilizando este primer método se deben deshabilitar las modificar los TRIGGERS de DUPLICATE-FIELD y DUPLICATE-RECORD para que no se copie el valor de la secuencia anterior en el nuevo registro.
- Otra manera, utilizando este método, es programar el TRIGGER PRE-INSERT de la siguiente manera.

```
SELECT nombre.NEXTVAL
INTO campo_secuencia
FROM DUAL;
```

- De esta manera se "gastan" menos números de la secuencia.
- El segundo método es utilizar una tabla de Secuencias propia del sistema que contenga los siguientes datos:

```
CREATE TABLE SECUENCIAS
(NOMBRE TABLA CHAR(40),
SECUENCIA NUMBER(5));
```

Y codificar los siguientes TRIGGERS

PRE-INSERT



```

SELECT SECUENCIA + 1, ROWID
INTO    :NOMBRE DEL BLOQUE.Nombre del campo, Campo clave
FROM    SECUENCIAS
WHERE   NOMBRE_TABLA = Nombre de la tabla
FOR UPDATE OF SECUENCIA;

```

Esta última línea previene la actualización del campo por otro usuario mientras el primer usuario no realice el update del campo SECUENCIA.

```

POST-INSERT
UPDATE SECUENCIAS
SET     SECUENCIA = :NOMBRE DEL BLOQUE.Nombre del campo
WHERE  ROWID = Campo clave;

```

La ventaja principal de este método es que se mantiene mayor control sobre los números generados y no se crean espacios entre la numeración.

5.6. VARIABLES

5.6.1. Uso de Variables en PL/SQL

- Cuando se creen variables en un PL/SQL siempre se debe definir el tamaño. Cada variable debe ser comentada para explicar su utilización.
- Si la variable se utiliza para almacenar datos de una tabla, se debe utilizar el atributo %TYPE para prevenir problemas o necesidad de modificación en el caso que se cambie la definición del campo en la tabla.

5.6.2. Uso de Variables Globales

- Las variables globales se utilizan para pasar datos entre formas y deben ser declaradas siempre utilizando el procedimiento DEFAULT_VALUE de manera de prevenir posibles errores en el caso que la variable no exista.
- Las variables globales se deben eliminar, utilizando el procedimiento RESET, luego que la variable sea utilizada. las variables globales se almacenan en memoria y pueden causar inconvenientes si no se eliminan luego de su utilización.

5.7. VIEWS

- Se debe evitar el uso de VIEWS para aplicaciones de FORMS. los VIEWS son ineficientes como para manejarlos en los QUERIES.

5.8. TABLA DUAL

- Se debe evitar el uso de la tabla DUAL con 2 excepciones: cuando se utiliza para generar "SEQUENCES" en un TRIGGER PRE-INSERT y para obtener la fecha del sistema (SYSDATE) y otros valores de la sesión (userid, termid, etc.).

5.9. CODIFICACIÓN PL/SQL

- Utilice programación PL/SQL siempre que se realicen operaciones que no involucren la utilización de tablas.
- La codificación con PL/SQL implica que se debe documentar extensamente para facilitar la revisión y mantenimiento.
- Siempre que se utilicen sentencias SELECT se debe manejar la excepción NO_DATA_FOUND, y si el WHERE no puede garantizar que se restrinja a un solo registro, entonces se debe manejar también la excepción TOO_MANY_ROWS.
- Se recomienda, adicionalmente, la utilización de la excepción WHEN OTHERS para manejar apropiadamente cualquier otra excepción no prevista con anterioridad y mostrar un mensaje entendible por el usuario de la siguiente manera:

```
EXCEPTION
```

```
    WHEN OTHERS THEN
```

```
        MESSAGE(PR_MENSAJE('AIF',NUMERO DEL  
MENSAJE,SQLCODE,' '));
```

```
        RAISE FORM_TRIGGER_FAILURE;
```

Siendo el número de mensaje el numero asignado a un mensaje en la tabla de mensajes con el siguiente texto:

```
"ha ocurrido un error en sistema, contacte a soporte.  
numero error &1"
```

- Evite la utilización de GOTO en un bloque PL/SQL.
- El DO_KEY se utiliza para simular la operación del usuario presionando la tecla correspondiente.

Por ejemplo, la sentencia DO_KEY('EXECUTE_QUERY') ejecuta el TRIGGER KEY-EXEQRY si es que existe, si no, se ejecuta el procedimiento EXECUTE_QUERY.

- Aproveche la facilidad que proveen los "PACKAGED PROCEDURES" SETFIELD, DISPLAYFIELD Y FIELDCHARACTERISTICS.

SETFIELD le cambia las características a un campo de la pantalla.

DISPLAYFIELD se utiliza para cambiar los atributos, dinámicamente, de un campo de la pantalla (por ejemplo cuando un valor esta dentro de un valor específico, se cambia el atributo de normal a video inverso. esto es de ayuda excepcional para el usuario) y

FIELDCHARACTERISTICS permite cambiar las características de edición del campo en pantalla (queryable, update, display, etc.).

- Cada vez que se realice un SELECT con el propósito de actualizar uno o varios de los campos seleccionados, se debe programar la instrucción de la siguiente manera:

```
SELECT CAMPO1, CAMPO2, ROWID  
    INTO VARIABLE1, VARIABLE2, CLAVE
```

```

FROM TABLA
WHERE CONDICIÓN
FOR UPDATE OF CAMPO1, CAMPO2;

```

y la instrucción de update de la siguiente manera:

```

UPDATE TABLA
SET CAMPO1 = VALOR1,
    CAMPO2 = VALOR2
WHERE ROWID = CLAVE;

```

Esto garantiza que entre el SELECT y el UPDATE otra aplicación no cambie los valores iniciales de los campos indicados en el FOR UPDATE OF.

- Los procedimientos comunes y referenciados por varias formas del sistema, deben ser almacenados en la forma **FRMACSEL** colocándole el nombre con el prefijo **FR_** de manera que se mantenga una librería de procedimientos claramente identificables en el sistema.
- Cada forma que utilice un procedimiento de la librería **LIBACSEL** debe ser registrado para que, en caso de modificaciones a estos procedimientos comunes, se sepa cuales son las formas que hay que regenerar.
- Los procedimientos y funciones almacenados en la base de datos comienzan siempre con el prefijo **PR_**. Revise si puede utilizar un procedimiento y / o función pre-programada antes de programar un nuevo procedimiento.
- En el caso que tenga que programar un procedimiento, considere la posibilidad de que sea utilizado por otras aplicaciones y agréguelo al **PACKED PROCEDURE** llamado **PR**.
- Antes de diseñar y construir una nueva aplicación, revise cuidadosamente los **TRIGGERS** de las tablas con las que la aplicación va a trabajar, para que los utilice y no genere código innecesario.

5.10. CURSORES

- Cuando se requiera hacer un SELECT para obtener un registro, siempre es más eficiente utilizar *cursores explícitos* en vez de *cursores implícitos*. por ejemplo, un cursor implícito seria:

```

SELECT Nombre
INTO      :B01.Nombre
FROM      EMPLEADO
WHERE     EMPLEADO.Codigo = :B01.Codigo
EXCEPTION
          WHEN NO_DATA_FOUND THEN
            NULL;

```

Si este QUERY es ejecutado 20 veces, ORACLE debe obtener 20 registros y realizar 40 FETCHES (20 para los datos y 20 para chequear la excepción TOO_MANY_ROWS).

En cambio, el mismo SELECT realizado de manera explícita, seria:



```
DECLARE
    CURSOR EMP IS
        SELECT Nombre
        FROM EMPLEADO
        WHERE EMPLEADO.Codigo = :B01.Codigo;
BEGIN
    FETCH EMP INTO :B01.Nombre;
    CLOSE EMP;
END;
```

- El ahorro en la cantidad de accesos que se hacen a la base de datos, es considerable y mientras mas registros se obtengan de esta manera, mayor será la eficiencia en la utilización de los recursos disponibles.